

```

import React, { useState, useEffect } from 'react';
import { Play, Pause, RotateCcw, Bell, CheckCircle, Shield } from 'lucide-react';

const VOCAB_DATABASE = [
  { day: 1, word: 'Nuance', phonetic: 'NOO-ahnss', def: 'Subtle difference in meaning', phrase: 'There is a real nuance here that changes our approach.' },
  { day: 2, word: 'Pragmatic', phonetic: 'prag-MAT-ik', def: 'Dealing with things practically', phrase: 'Let us take a pragmatic view of what we can finish.' },
  { day: 3, word: 'Ambiguity', phonetic: 'am-bih-GYOO-ih-tee', def: 'Lack of exactness', phrase: 'We need to eliminate ambiguity in our roles.' },
  { day: 4, word: 'Catalyst', phonetic: 'KAT-uh-list', def: 'Event that accelerates change', phrase: 'That feedback was the catalyst for our redesign.' },
  { day: 5, word: 'Compelling', phonetic: 'kum-PELL-ing', def: 'Powerfully convincing', phrase: 'You make a compelling argument.' },
  { day: 6, word: 'Efficacy', phonetic: 'EF-ih-kuh-see', def: 'Ability to produce intended result', phrase: 'Let us measure the actual efficacy of this strategy.' },
  { day: 7, word: 'Paradigm', phonetic: 'PAIR-uh-dime', def: 'Typical pattern or framework', phrase: 'This represents a complete paradigm shift.' },
  { day: 8, word: 'Acutely', phonetic: 'uh-KYOOT-lee', def: 'Intensely aware or perceptive', phrase: 'I am acutely aware of the timeline pressures.' },
  { day: 9, word: 'Pivotal', phonetic: 'PIV-uh-tul', def: 'Of crucial importance', phrase: 'That choice was the pivotal moment for progress.' },
  { day: 10, word: 'Disparate', phonetic: 'DIS-pur-it', def: 'Essentially different in kind', phrase: 'We are bringing these disparate ideas together.' },
  { day: 11, word: 'Incisive', phonetic: 'in-SIGH-siv', def: 'Intelligently analytical and clear', phrase: 'Thank you for that incisive observation.' },
  { day: 12, word: 'Robust', phonetic: 'roh-BUST', def: 'Sturdy and resilient', phrase: 'We need a more robust framework before scaling.' },
  { day: 13, word: 'Tacit', phonetic: 'TASS-it', def: 'Understood without being spoken', phrase: 'There was a tacit agreement in the room.' },
  { day: 14, word: 'Tenacious', phonetic: 'tuh-NAY-shuss', def: 'Persistent and firm', phrase: 'They were tenacious in pursuing that resolution.' }
];

export default function S25ArticulationApp() {
  const [activeDay, setActiveDay] = useState(1);
  const [secondsRemaining, setSecondsRemaining] = useState(540); // 9 minutes = 540s
  const [isActive, setIsActive] = useState(false);
  const [streak, setStreak] = useState(0);
  const [reminderTime, setReminderTime] = useState('08:30');

  useEffect(() => {
    let interval = null;
    if (isActive && secondsRemaining > 0) {

```

```

    interval = setInterval(() => setSecondsRemaining(prev => prev - 1), 1000);
  } else if (secondsRemaining === 0 && isActive) {
    setIsActive(false);
    setStreak(prev => prev + 1);
    alert('Training complete! Great work maintaining your vocal streak.');
```

```
  }
```

```
  return () => clearInterval(interval);
```

```
}, [isActive, secondsRemaining]);
```

```
const currentVocab = VOCAB_DATABASE[(activeDay - 1) % 14];
```

```
const getActivePhase = () => {
```

```
  const elapsed = 540 - secondsRemaining;
```

```
  if (elapsed < 120) return { num: 1, title: 'Phase 1: Physical Warm-Up', detail: '0:00-0:45 Lip Trills | 0:45-1:30 Tongue Circles | 1:30-2:00 Jaw Release' };
```

```
  if (elapsed < 300) return { num: 2, title: 'Phase 2: Target Drills & Vocab', detail: `Practice: Pah-Bah-Mah / Tah-Dah-Nah. Daily Word: ${currentVocab.word} (${currentVocab.phonetic})` };
```

```
  if (elapsed < 420) return { num: 3, title: 'Phase 3: Speed & Pencil Drill', detail: 'Place a pen horizontally between teeth. Read out loud for 60 seconds with crisp articulation.' };
```

```
  return { num: 4, title: 'Phase 4: Real-World Roleplay', detail: `Speak out loud: "${currentVocab.phrase}" using a deliberate downward inflection.` };
```

```
};
```

```
const formatTime = (secs) => {
```

```
  const m = Math.floor(secs / 60);
```

```
  const s = secs % 60;
```

```
  return `${m}:${s < 10 ? '0' : ''}${s}`;
```

```
};
```

```
const requestNotification = () => {
```

```
  if ('Notification' in window) {
```

```
    Notification.requestPermission().then(permission => {
```

```
      if (permission === 'granted') {
```

```
        alert(`Daily reminder scheduled local-only for ${reminderTime}`);
```

```
      }
```

```
    });
```

```
  }
```

```
};
```

```
const phase = getActivePhase();
```

```
return (
```

```
  <div className="min-h-screen bg-neutral-950 text-white font-sans p-4 max-w-md mx-auto flex flex-col justify-between border-x border-neutral-800">
```

```

    {/ * Header */}
    <div className="flex justify-between items-center pb-4 border-b border-neutral-800">
      <div>
        <h1 className="text-xl font-bold tracking-wide text-indigo-400">Articulation 9M</h1>
        <p className="text-xs text-neutral-400 flex items-center gap-1">
          <Shield className="w-3 h-3 text-emerald-400" /> S25 Local Secure Environment
        </p>
      </div>
      <div className="bg-neutral-900 border border-neutral-800 px-3 py-1 rounded-full text-xs font-medium text-emerald-400">
        Streak: {streak} Days
      </div>
    </div>

    {/ * Main Timer Display */}
    <div className="my-6 text-center bg-neutral-900 border border-neutral-800 rounded-2xl p-6 shadow-xl">
      <div className="text-6xl font-black tracking-tight text-white font-mono mb-2">
        {formatTime(secondsRemaining)}
      </div>
      <div className="text-sm font-semibold text-indigo-400 mb-1">{phase.title}</div>
      <p className="text-xs text-neutral-300 leading-relaxed min-h-[40px] px-2">{phase.detail}</p>

      {/ * Timer Controls */}
      <div className="flex justify-center gap-4 mt-6">
        <button
          onClick={() => setIsActive(!isActive)}
          className="flex items-center gap-2 bg-indigo-600 hover:bg-indigo-500 text-white font-semibold px-6 py-3 rounded-xl transition"
        >
          {isActive ? <Pause className="w-5 h-5" /> : <Play className="w-5 h-5" />}
          {isActive ? 'Pause' : 'Start Training'}
        </button>
        <button
          onClick={() => { setIsActive(false); setSecondsRemaining(540); }}
          className="p-3 bg-neutral-800 hover:bg-neutral-700 text-neutral-300 rounded-xl transition"
        >
          <RotateCcw className="w-5 h-5" />
        </button>
      </div>
    </div>

```

```

    { /* Daily Vocabulary Focus Card */
    <div className="bg-neutral-900/80 border border-neutral-800 rounded-xl p-4 mb-4">
      <div className="flex justify-between items-center mb-2">
        <span className="text-xs font-bold uppercase tracking-wider text-neutral-400">Day
{activeDay} Vocabulary Focus</span>
        <select
          value={activeDay}
          onChange={(e) => setActiveDay(Number(e.target.value))}
          className="bg-neutral-800 text-xs text-neutral-200 border border-neutral-700 rounded
px-2 py-1"
        >
          {VOCAB_DATABASE.map(v => (
            <option key={v.day} value={v.day}>Day {v.day}: {v.word}</option>
          ))}
        </select>
      </div>
      <div className="text-lg font-bold text-white">{currentVocab.word} <span
className="text-xs font-normal text-indigo-300">({currentVocab.phonetic})</span></div>
      <div className="text-xs text-neutral-400 italic mb-2">{currentVocab.def}</div>
      <div className="bg-neutral-950 p-2 rounded border border-neutral-800/60 text-xs
text-emerald-300">
        "{currentVocab.phrase}"
      </div>
    </div>

    { /* Security & Reminder Settings */
    <div className="bg-neutral-900/50 border border-neutral-800/80 rounded-xl p-3 text-xs flex
justify-between items-center">
      <div className="flex items-center gap-2">
        <Bell className="w-4 h-4 text-indigo-400" />
        <div>
          <div className="font-medium text-neutral-200">Daily Alarm</div>
          <input
            type="time"
            value={reminderTime}
            onChange={(e) => setReminderTime(e.target.value)}
            className="bg-neutral-800 text-neutral-300 rounded px-1 mt-0.5"
          />
        </div>
      </div>
      <button
        onClick={requestNotification}
        className="bg-neutral-800 hover:bg-neutral-700 border border-neutral-700 px-3 py-1.5
rounded-lg font-medium text-neutral-200 transition"

```

```
>  
  Set Reminder  
</button>  
</div>  
</div>  
);  
}
```